4. HAFTA: BİLGİSAYAR MİMARİSİ — DERS NOTU

1. GİRİŞ

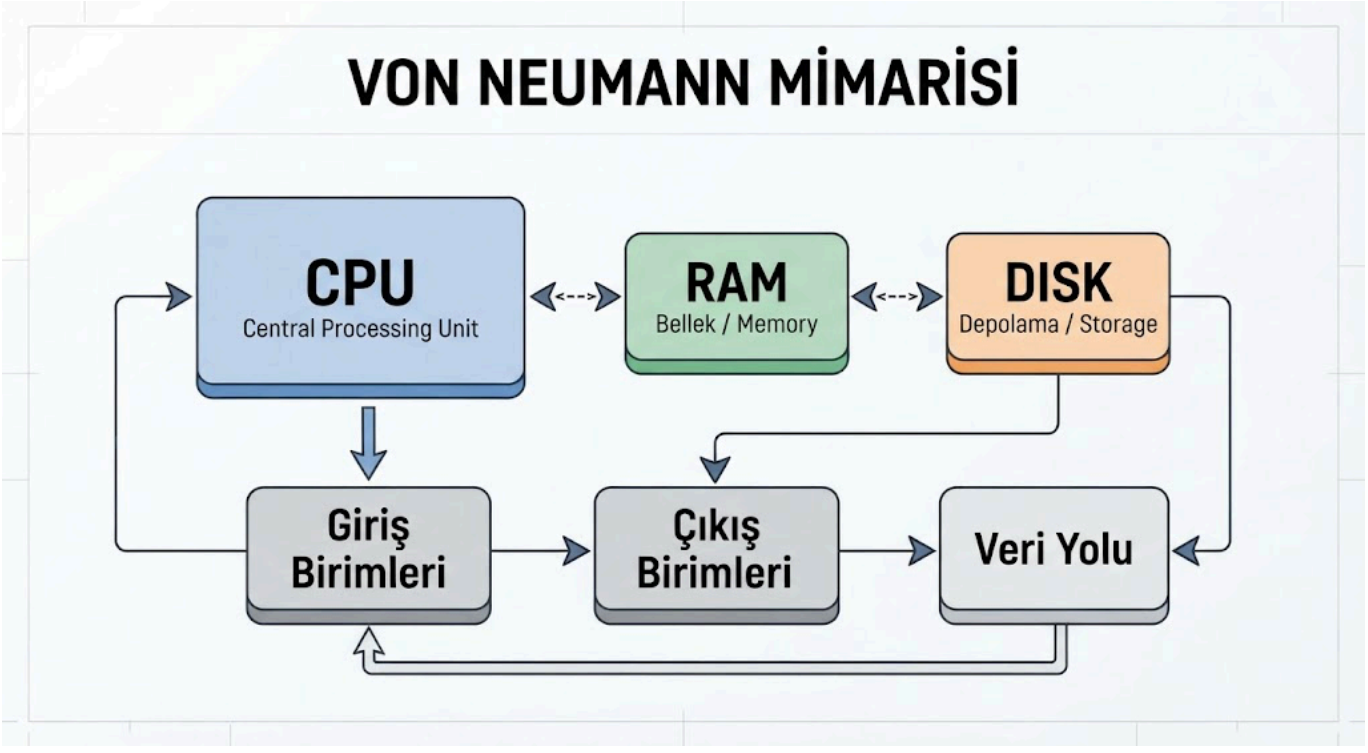
Bir program yazıldığında, bu programın bilgisayarda nasıl çalıştığını anlamak için bilgisayar mimarisinin temel bileşenlerini bilmek gerekir. Bu bölümde bilgisayarın temel donanım birimleri (CPU, RAM, Disk), bir komutun işlenme süreci (Fetch–Decode–Execute) ve programların makine diline nasıl çevrildiği (derleyici ve yorumlayıcı) ele alınacaktır. Son olarak bir programın çalışma süreci baştan sona açıklanacaktır.

2. BİLGİSAYAR MİMARİSİNE GENEL BAKIŞ

Bilgisayar, **donanım (hardware)** ve **yazılım (software)** olmak üzere iki temel bileşenden oluşur. Donanım fiziksel parçaları, yazılım ise bu parçalar üzerinde çalışan programları ifade eder.

2.1 Von Neumann Mimarisi

Modern bilgisayarların çoğu **Von Neumann mimarisine** dayanır. Bu mimarinin temel özelliği, **veri ve komutların aynı bellekte saklanmasıdır**.

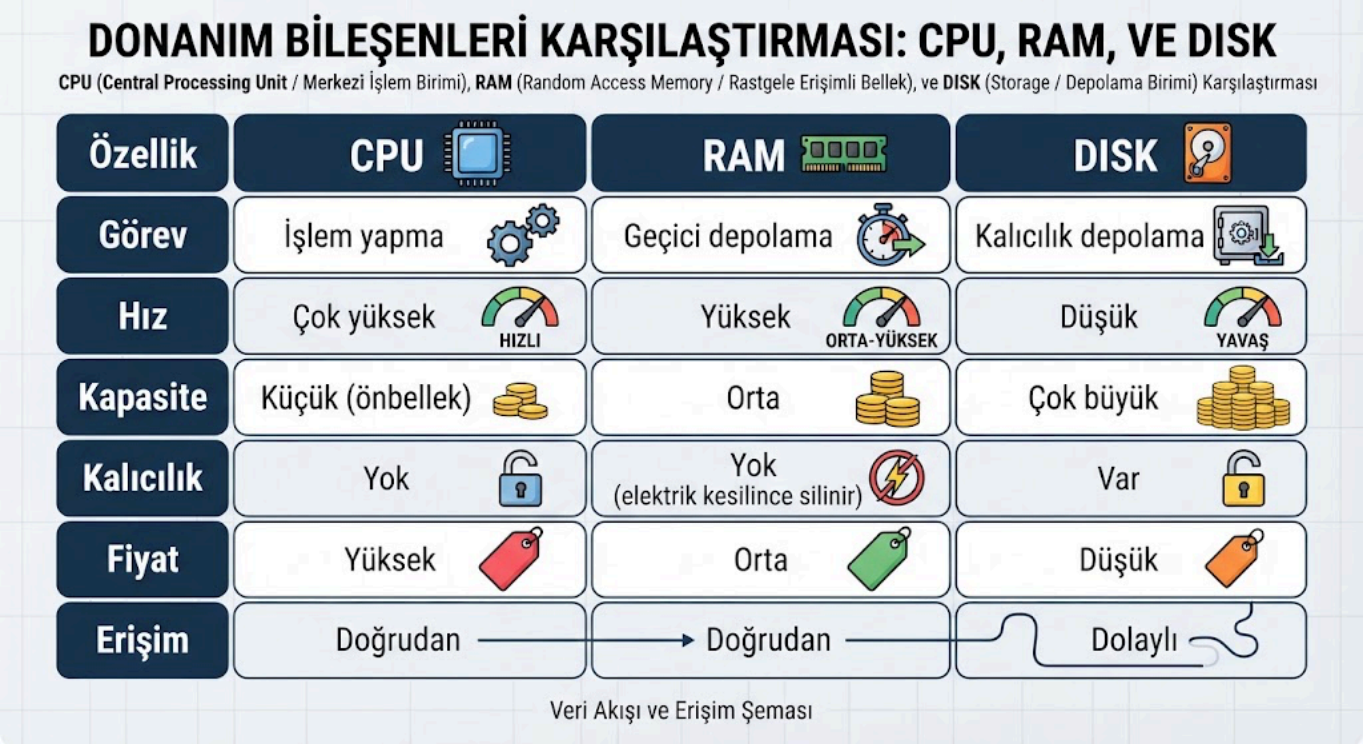


Temel Bileşenler:

Bileşen	Görevi
CPU	Komutları işler, hesaplamaları yapar
RAM	Çalışan programları ve verileri geçici olarak tutar
Disk	Kalıcı veri depolama

Bileşen	Görevi
Giriş Birimleri	Klavye, fare, tarayıcı
Çıkış Birimleri	Ekran, yazıcı, hoparlör
Veri Yolu (Bus)	Bileşenler arası veri iletişimi

2.2 Temel Bileşenlerin Karşılaştırması

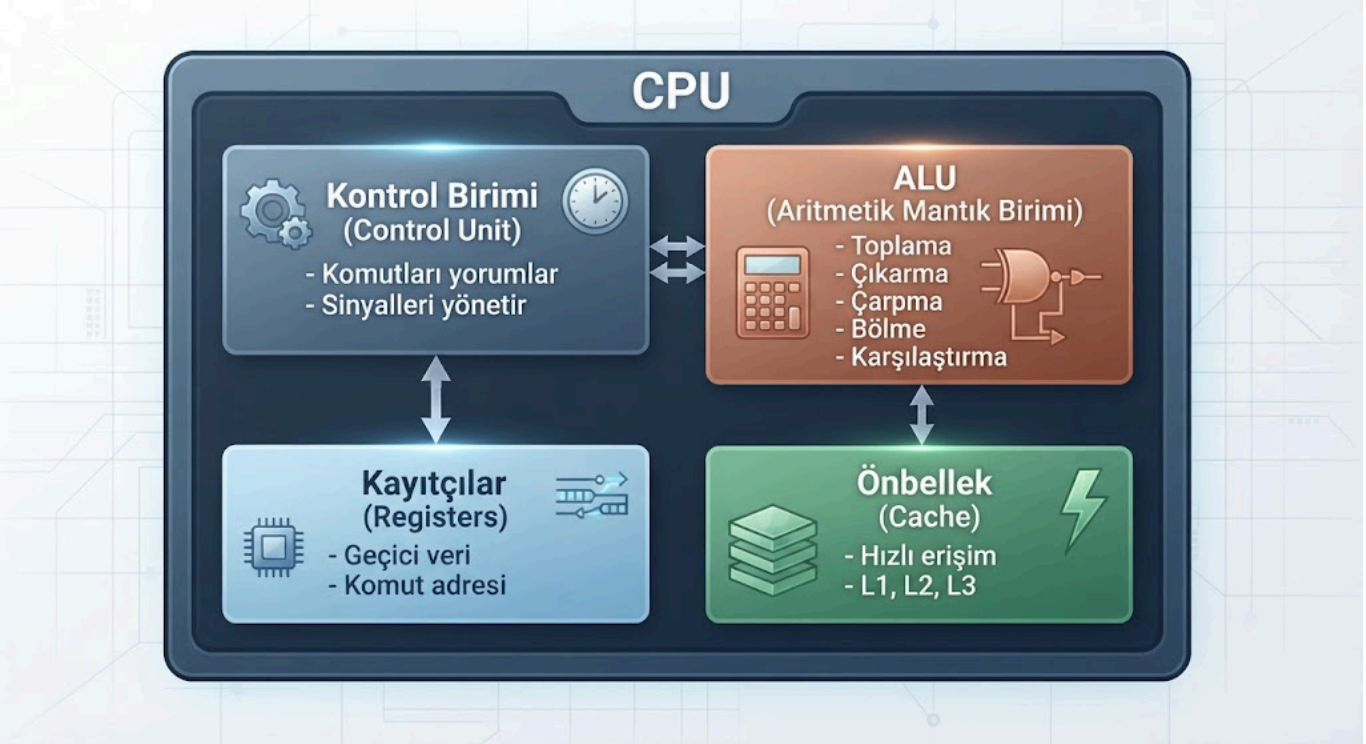


---

3. CPU (MERKEZİ İŞLEM BİRİMİ)

CPU (Central Processing Unit), bilgisayarın beyni olarak adlandırılır. Tüm hesaplamalar ve komut işlemleri burada gerçekleşir.

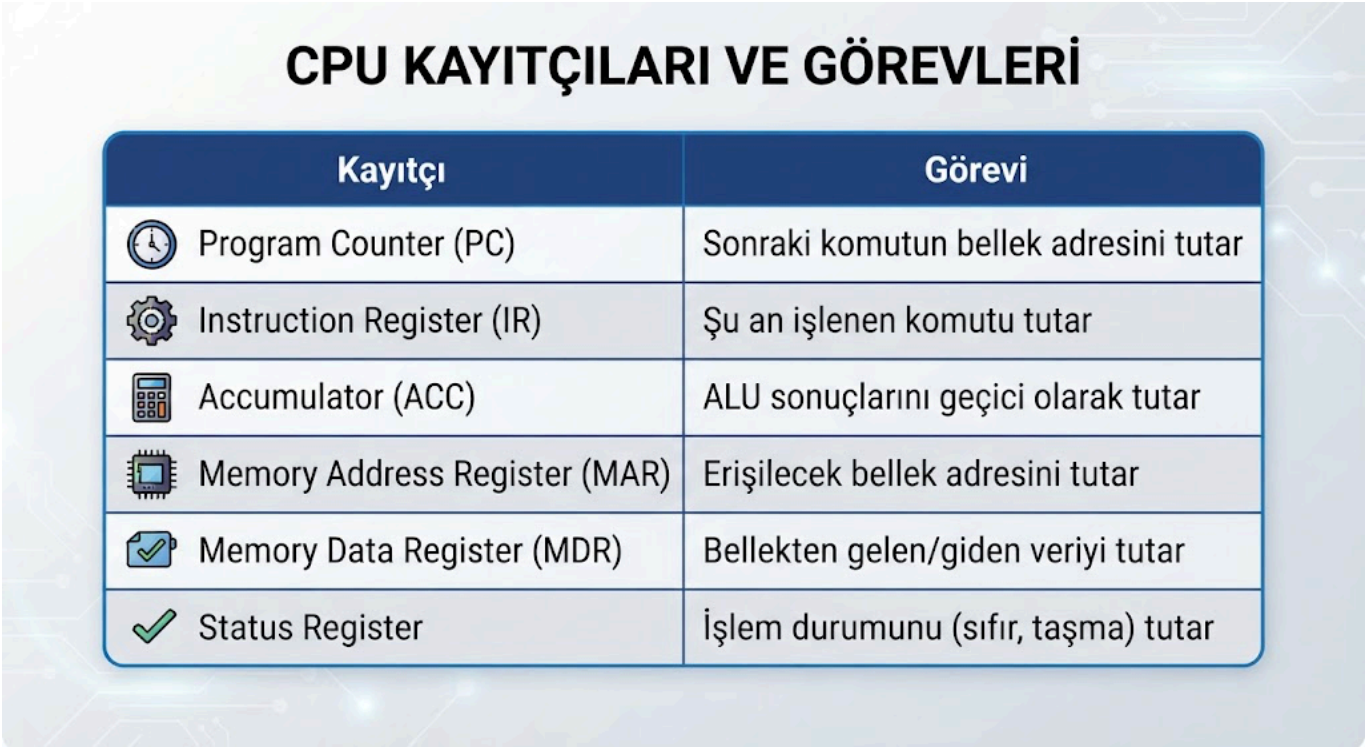
3.1 CPU'nun Temel Bileşenleri



3.2 CPU Bileşenlerinin Görevleri

Bileşen	Görevi
Kontrol Birimi (CU)	Komutları alır, çözümler ve diğer birimlere sinyal gönderir
ALU (Aritmetik Mantık Birimi)	Matematiksel ve mantıksal işlemleri yapar
Kayıtçılar (Registers)	CPU içindeki en hızlı geçici depolama alanları
Önbellek (Cache)	RAM'e göre daha hızlı, sık kullanılan verileri tutar
Saat (Clock)	İşlem hızını belirleyen sinyal üreteci

3.3 Kayıtçı Türleri



3.4 CPU Hızını Etkileyen Faktörler

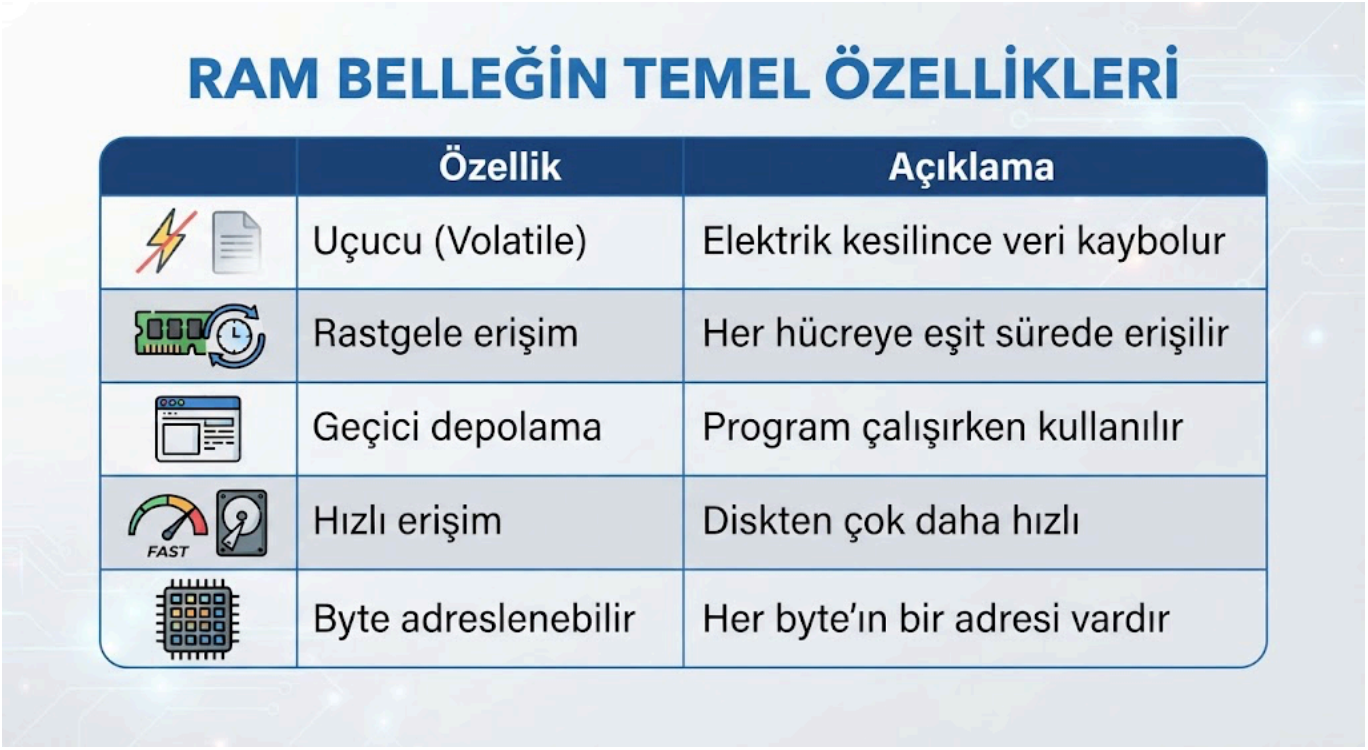
1. **Saat Hızı (Clock Speed):** GHz cinsinden ölçülür. Saniyedeki işlem sayısını belirler.
2. **Çekirdek Sayısı (Core Count):** Aynı anda kaç işlem yapılabileceğini belirler.
3. **Önbellek Boyutu:** Sık kullanılan verilerin hızlı erişimini sağlar.
4. **Komut Seti Mimarisi:** RISC veya CISC olması performansı etkiler.
5. **Veri Yolu Genişliği:** Aynı anda taşınabilecek bit sayısı.

---

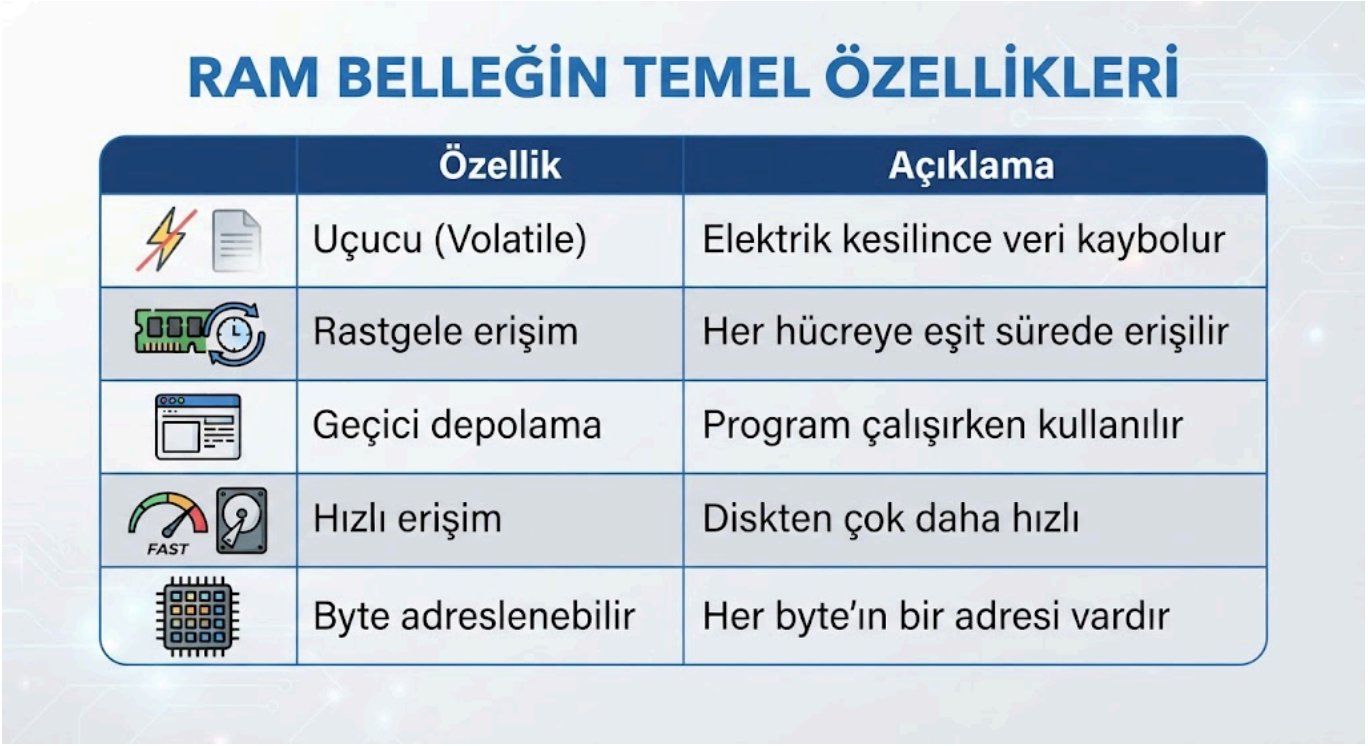
4. RAM (RASTGELE ERİŞİMLİ BELLEK)

RAM (Random Access Memory), çalışan programların ve verilerin geçici olarak saklandığı yerdir. Elektrik kesildiğinde içeriği silinir (uçucu bellek).

4.1 RAM'in Temel Özellikleri

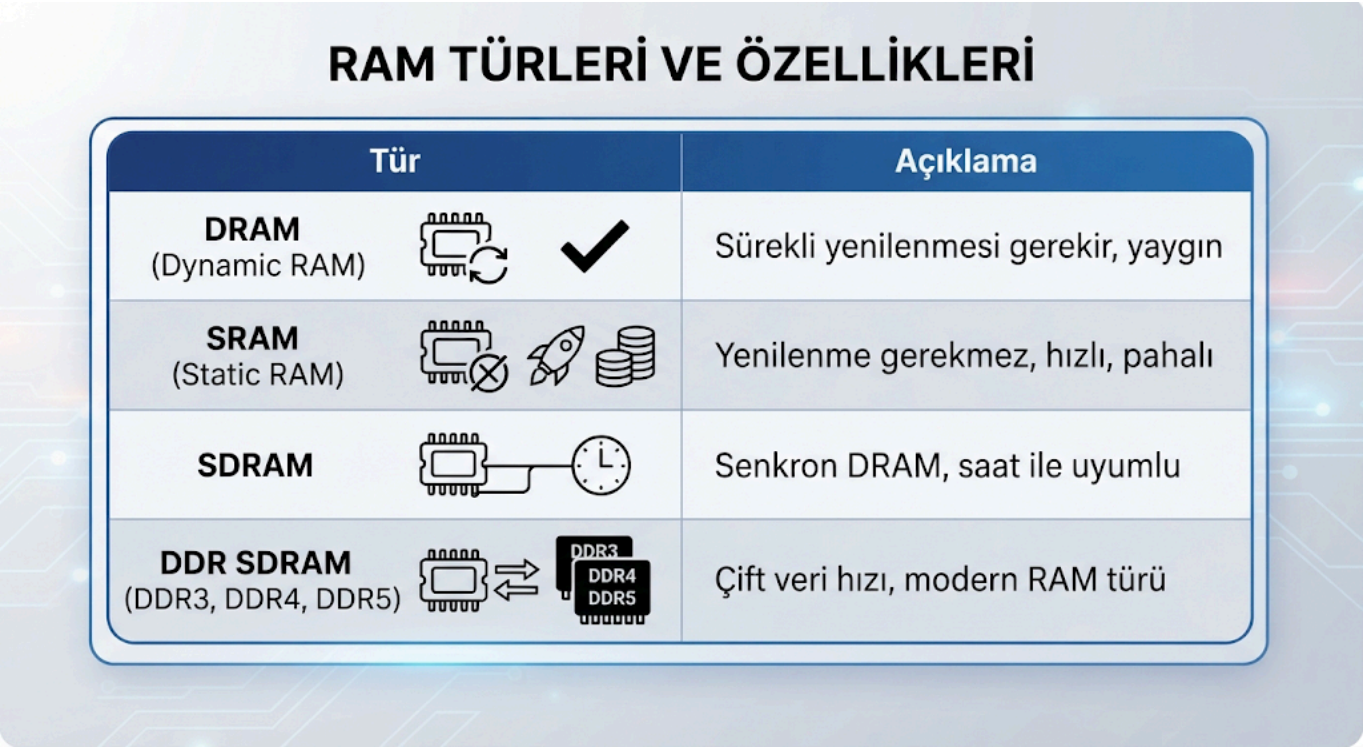


4.2 RAM Türleri



4.3 Bellek Hiyerarşisi

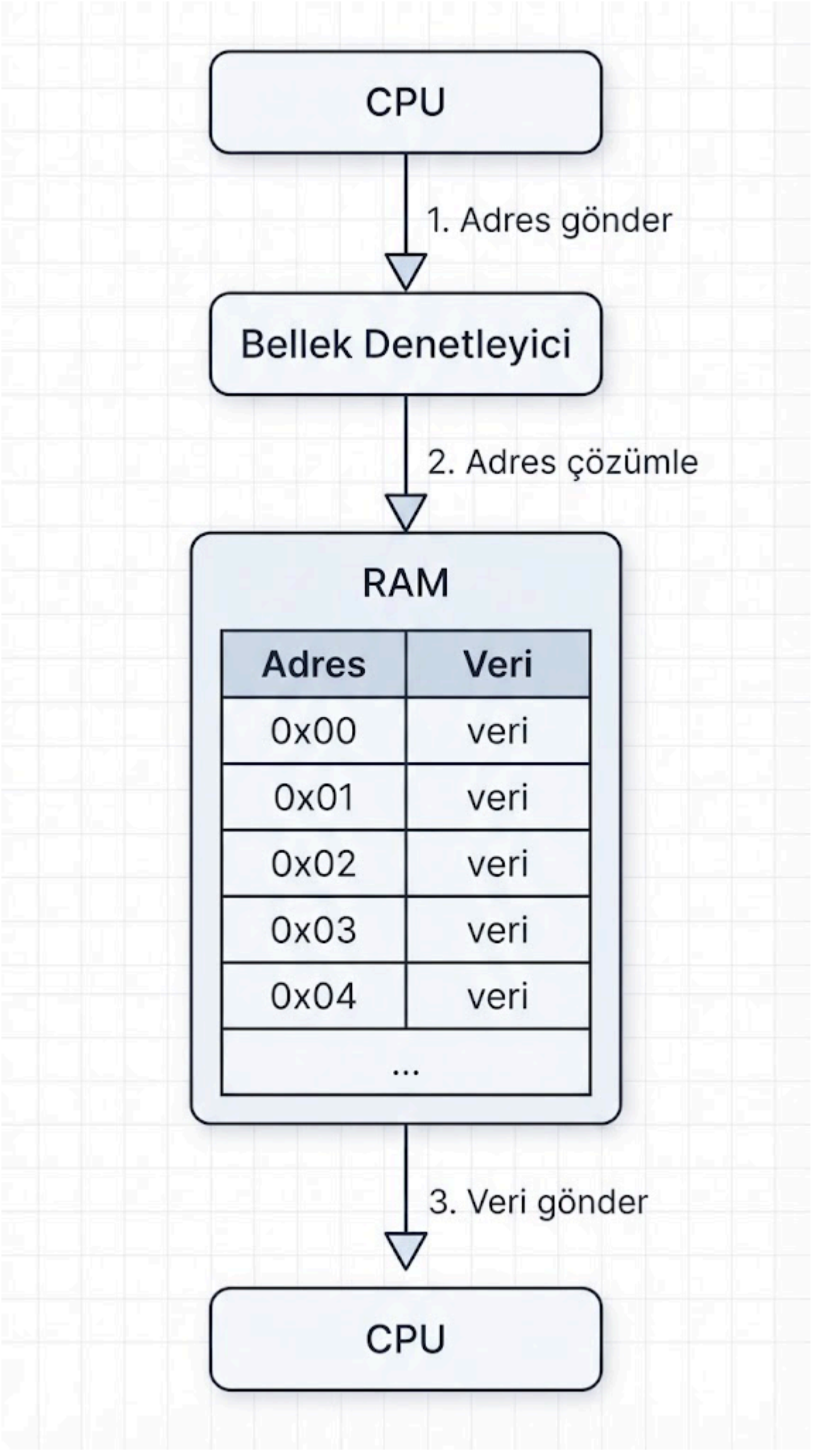
Bilgisayarda bellek, hız ve kapasite açısından katmanlı bir yapı gösterir.



Hiyerarşinin Mantığı:

- CPU'ya yakın olan bellek daha hızlı ama daha küçük ve pahalıdır.
- CPU'dan uzaklaştıkça kapasite artar, hız düşer, fiyat azalır.
- Amaç: Sık kullanılan veriyi hızlı katmanda tutarak performansı artırmak.

4.4 RAM'in Çalışma Prensibi


CPU
1. Adres gönder
Bellek Denetleyici
2. Adres çözümle
RAM
Adres
Veri
0x00
veri
0x01
veri
0x02
veri
0x03
veri
0x04
veri
...
3. Veri gönder
CPU

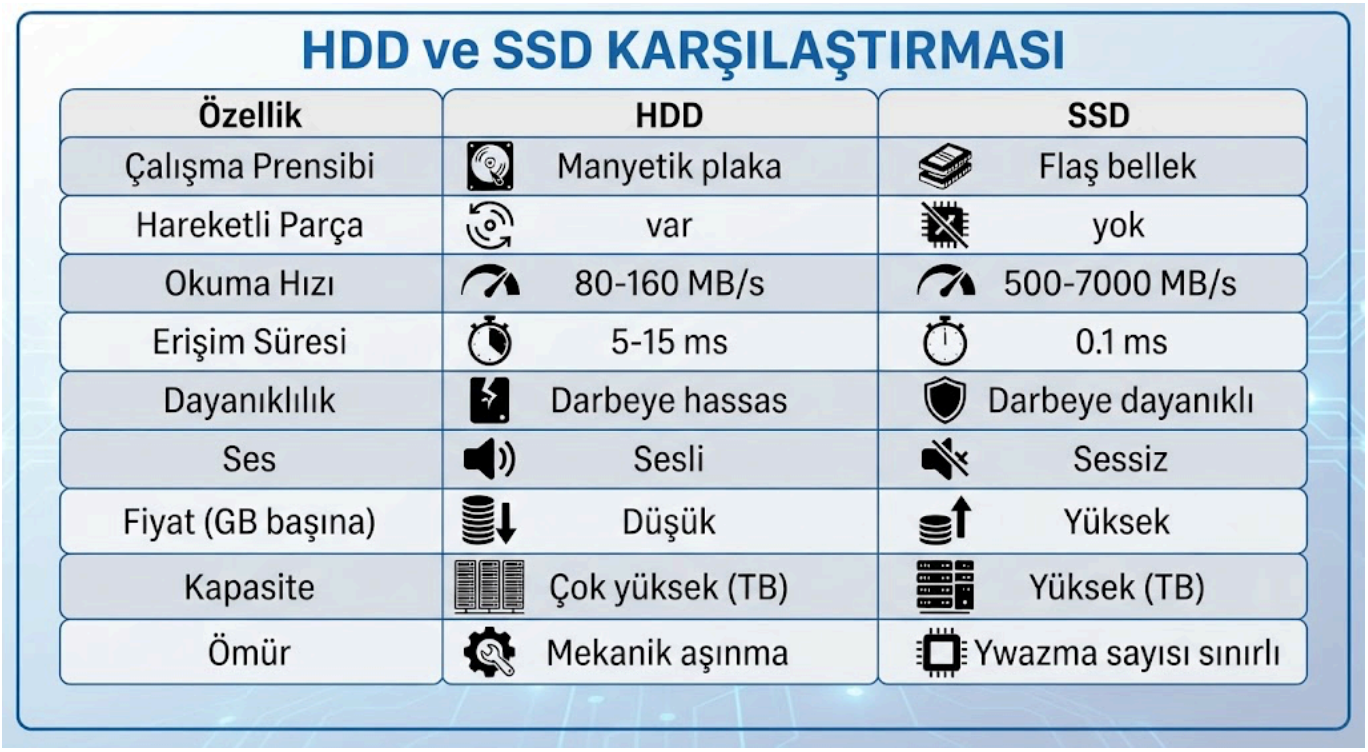
5. DİSK (KALICI DEPOLAMA)

Disk, verilerin kalıcı olarak saklandığı depolama birimidir. Elektrik kesildiğinde veriler korunur (uçucu olmayan bellek).

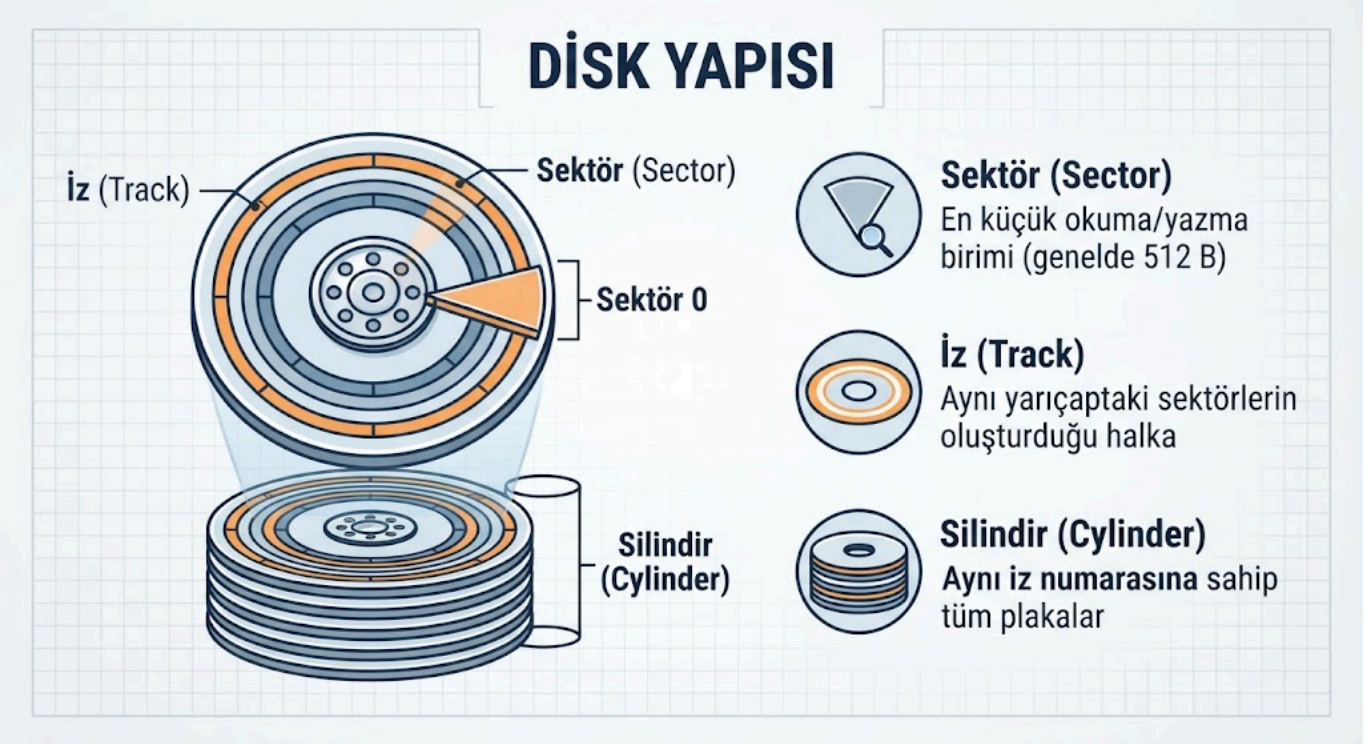
5.1 Disk Türleri



5.2 HDD vs SSD Karşılaştırması



5.3 Disk Yapısı



5.4 Dosya Sistemi

Disk üzerindeki verilerin düzenlenmesini sağlayan yapıya **dosya sistemi** denir.

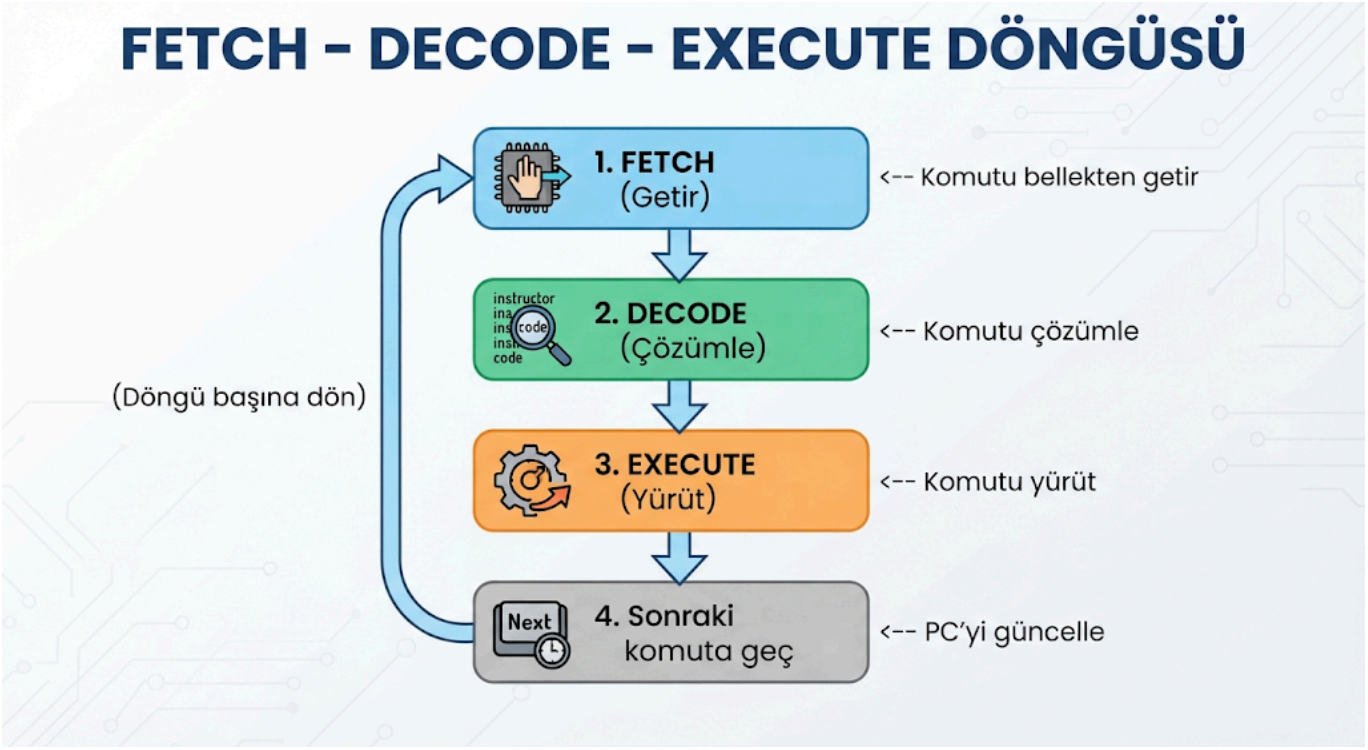
Dosya Sistemi	Kullanım Alanı
FAT32	Eski Windows, USB bellekler
NTFS	Modern Windows
ext4	Linux
APFS	macOS
exFAT	Taşınabilir diskler

---

6. FETCH–DECODE–EXECUTE DÖNGÜSÜ

CPU, bir programı çalıştırırken her komutu üç temel aşamada işler: **Fetch (Getir)**, **Decode (Çözümle)**, **Execute (Yürüt)**. Bu döngü, program bitene kadar devam eder.

6.1 Döngünün Genel Yapısı

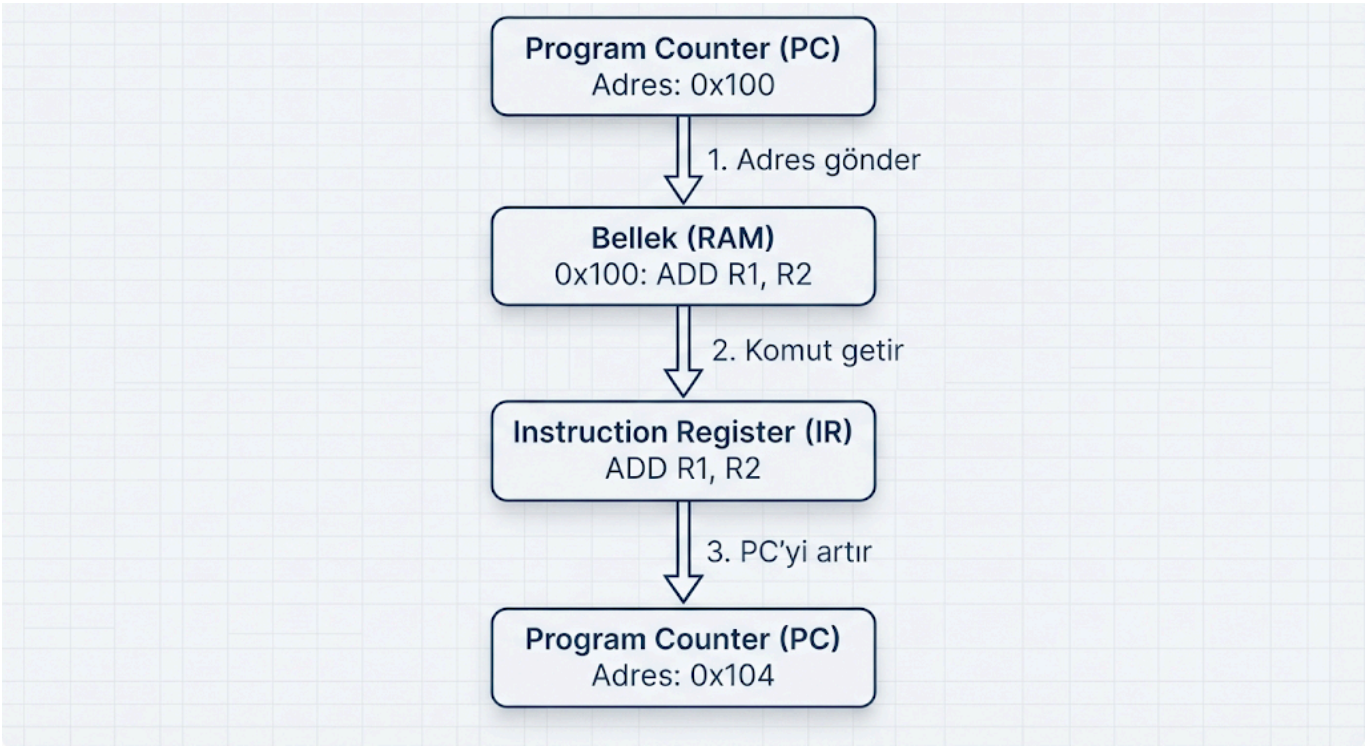


6.2 Fetch (Getir) Aşaması

Bu aşamada CPU, **Program Counter (PC)**'ın gösterdiği adresteki komutu bellekten alır.

Adımlar:

1. PC'deki adres, MAR'a (Memory Address Register) kopyalanır.
2. MAR'daki adres, bellek veri yoluna gönderilir.
3. Bellekten gelen komut, MDR'ye (Memory Data Register) yüklenir.
4. MDR'deki komut, IR'ye (Instruction Register) kopyalanır.
5. PC, bir sonraki komutun adresini gösterecek şekilde artırılır.

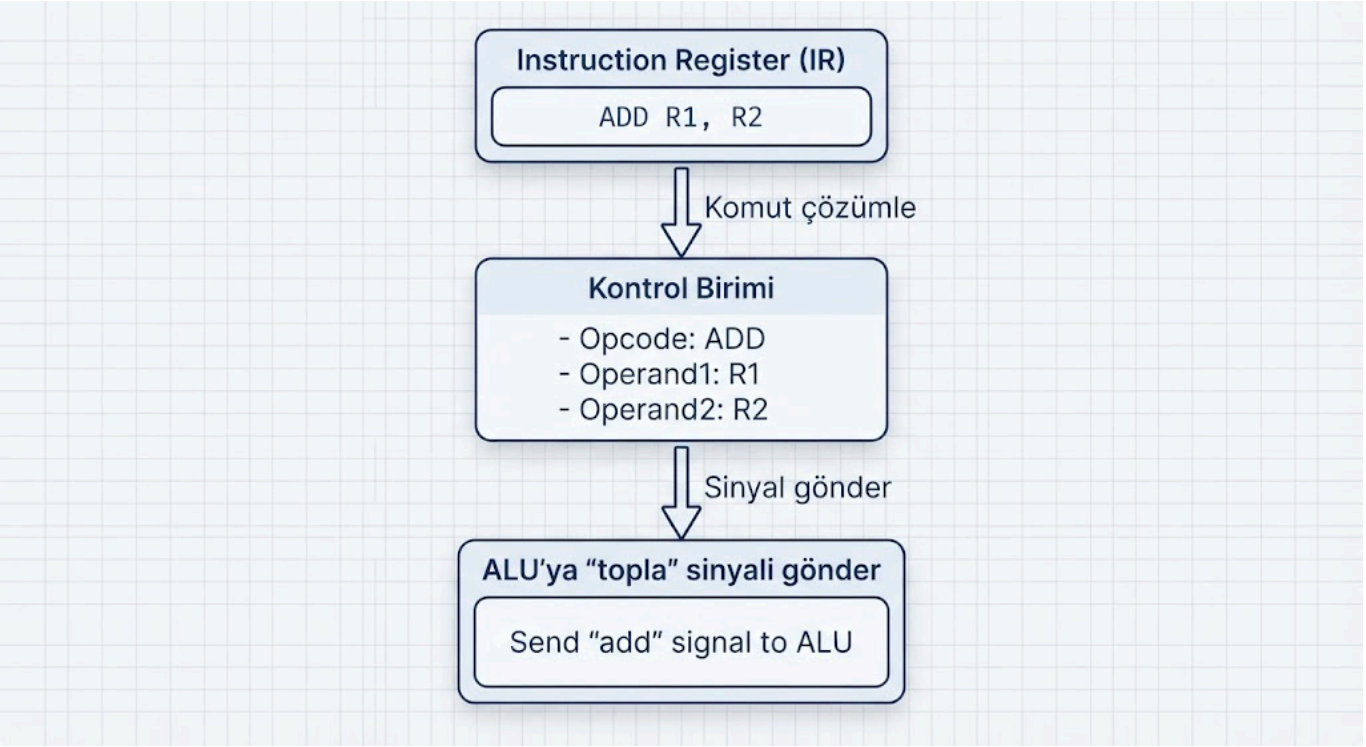


6.3 Decode (Çözümle) Aşaması

Bu aşamada **Kontrol Birimi**, IR'deki komutu çözümler ve hangi işlemin yapılacağını belirler.

Yapılan İşlemler:

1. Komut türü belirlenir (toplama, çıkarma, yükleme, atlama vb.).
2. Gerekli operandlar (veriler) tespit edilir.
3. İlgili birimlere (ALU, bellek, kayıtçılar) sinyal gönderilir.

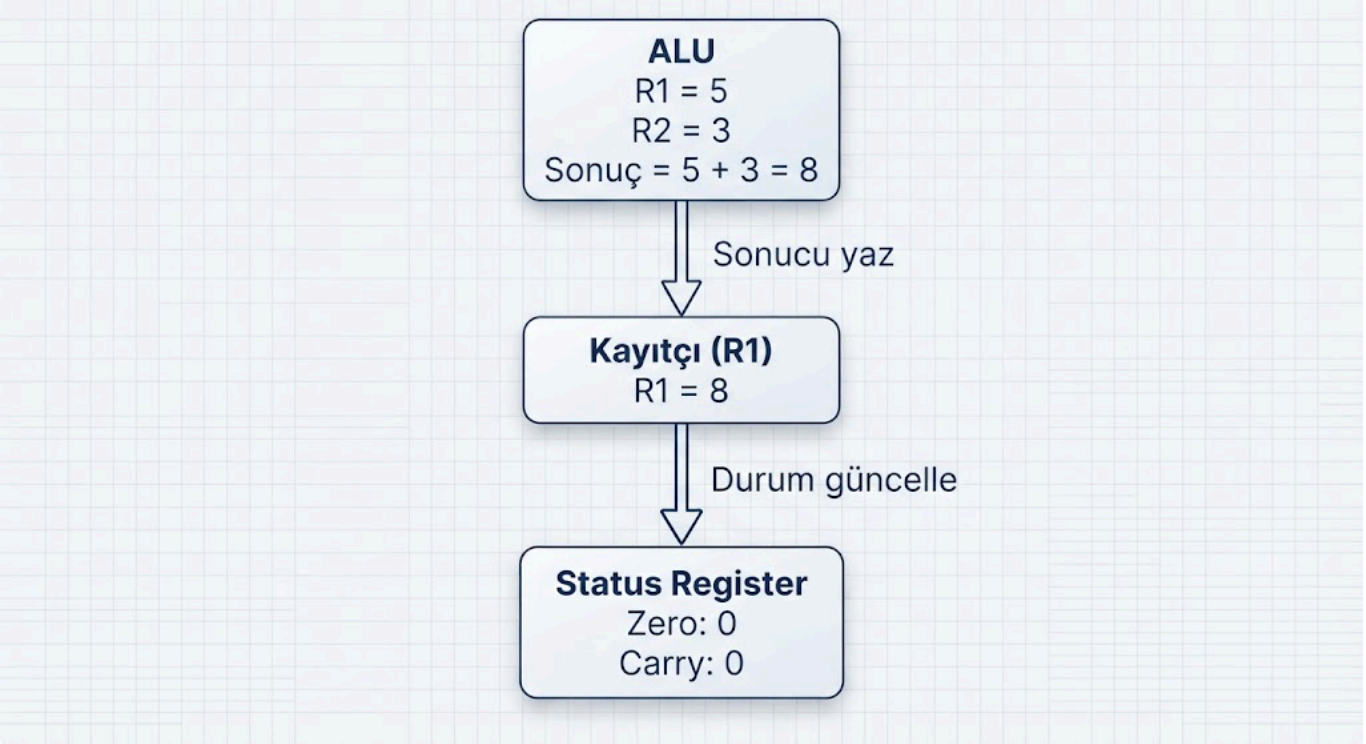


6.4 Execute (Yürüt) Aşaması

Bu aşamada komut fiilen gerçekleştirilir.

Yapılan İşlemler:

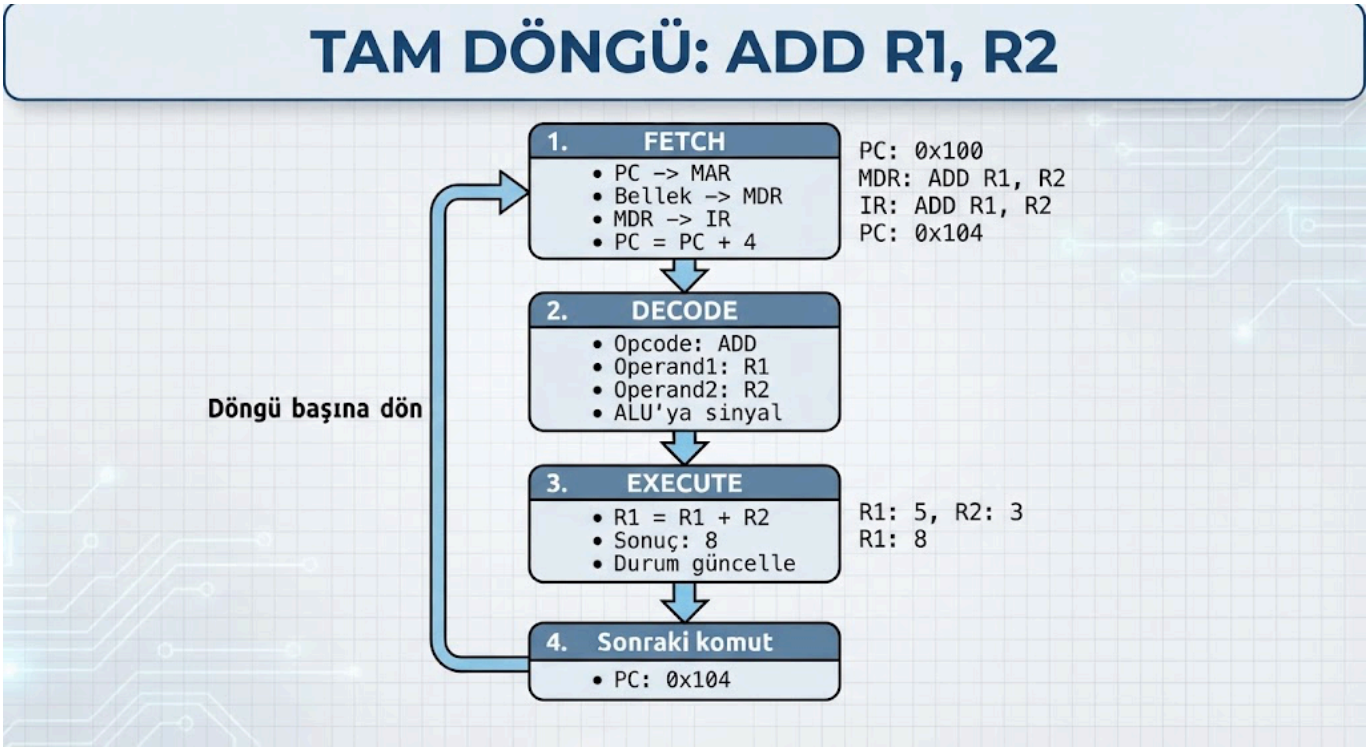
1. ALU, gerekli hesaplamayı yapar.
2. Sonuç, bir kayıtçıya veya belleğe yazılır.
3. Durum kayıtçısı (Status Register) güncellenir.
4. Gerekirse PC değiştirilir (atlama komutlarında).



6.5 Tam Döngü Örneği

Aşağıda basit bir komutun baştan sona işlenmesi gösterilmiştir.

Komut: `ADD R1, R2` (R1 ve R2'yi topla, sonucu R1'e yaz)



6.6 Döngünün Süresi

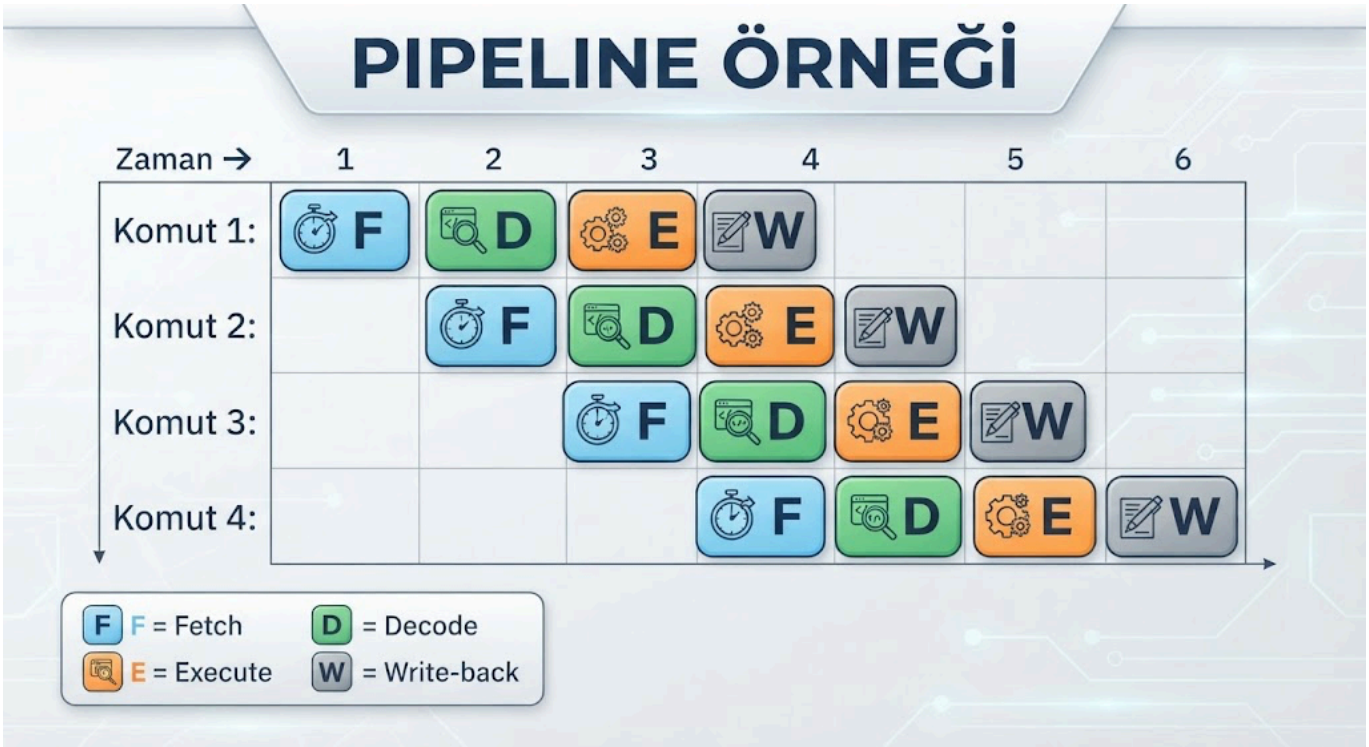
Her aşama belirli sayıda saat çevrimi (clock cycle) gerektirir.



Örnek: 3 GHz'lik bir CPU'da saniyede 3 milyar saat çevrimi vardır. Ortalama bir komut 4 çevrimde tamamlanıyorsa, saniyede yaklaşık 750 milyon komut işlenebilir.

6.7 Pipeline (Boruhattı) Kavramı

Modern CPU'lar, Fetch–Decode–Execute aşamalarını **paralel** olarak yürütür. Buna **pipeline** denir.



Pipeline sayesinde her saat çevriminde bir komut tamamlanabilir; bu da performansı önemli ölçüde artırır.

---

7. DERLEYİCİ vs YORUMLAYICI

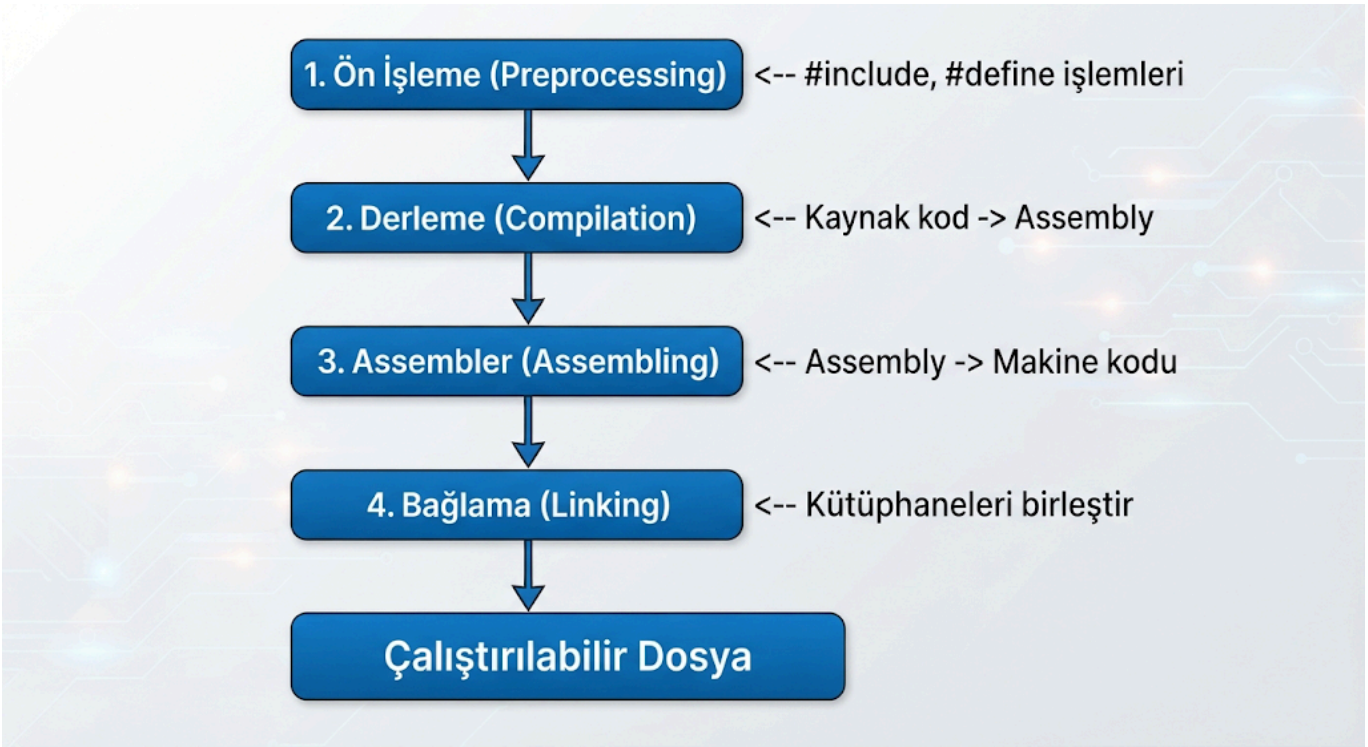
Yazılan programlar (kaynak kod) doğrudan CPU tarafından çalıştırılamaz. Bunların makine diline çevrilmesi gerekir. Bu çeviri işlemi iki temel yöntemle yapılır: **derleme (compilation)** ve **yorumlama (interpretation)**.

7.1 Derleyici (Compiler)

Derleyici, kaynak kodun tamamını baştan sona okuyup **tek seferde** makine diline çeviren programdır. Çıktı olarak çalıştırılabilir bir dosya üretir.



Derleyici Çalışma Aşamaları:



Derleyici Avantajları:

- Çalışma hızı yüksektir (kod önceden çevrilmiştir).
- Kaynak kod gizli kalabilir (yalnızca makine kodu dağıtılır).
- Hata ayıklama derleme sırasında yapılabilir.
- Optimizasyon yapılabilir.

Derleyici Dezavantajları:

- Her platform için ayrı derleme gerekir.
- Derleme süresi uzun olabilir.
- Kod değişikliğinde yeniden derleme gerekir.

Örnek Diller: C, C++, Rust, Go, Pascal

7.2 Yorumlayıcı (Interpreter)

Yorumlayıcı, kaynak kodu satır satır okuyup **anında** çalıştıran programdır. Ara makine kodu dosyası üretmez.



Yorumlayıcı Avantajları:

- Platform bağımsızdır (yorumlayıcı varsa çalışır).
- Kod değişikliğinde yeniden derleme gerekmez.
- Hata ayıklama kolaydır.
- Geliştirme hızlıdır.

Yorumlayıcı Dezavantajları:

- Çalışma hızı düşüktür (her satır anında çevrilir).
- Kaynak kod açıktır.
- Her çalıştırmada çeviri yapılır.

Örnek Diller: Python, Ruby, JavaScript, PHP, Perl

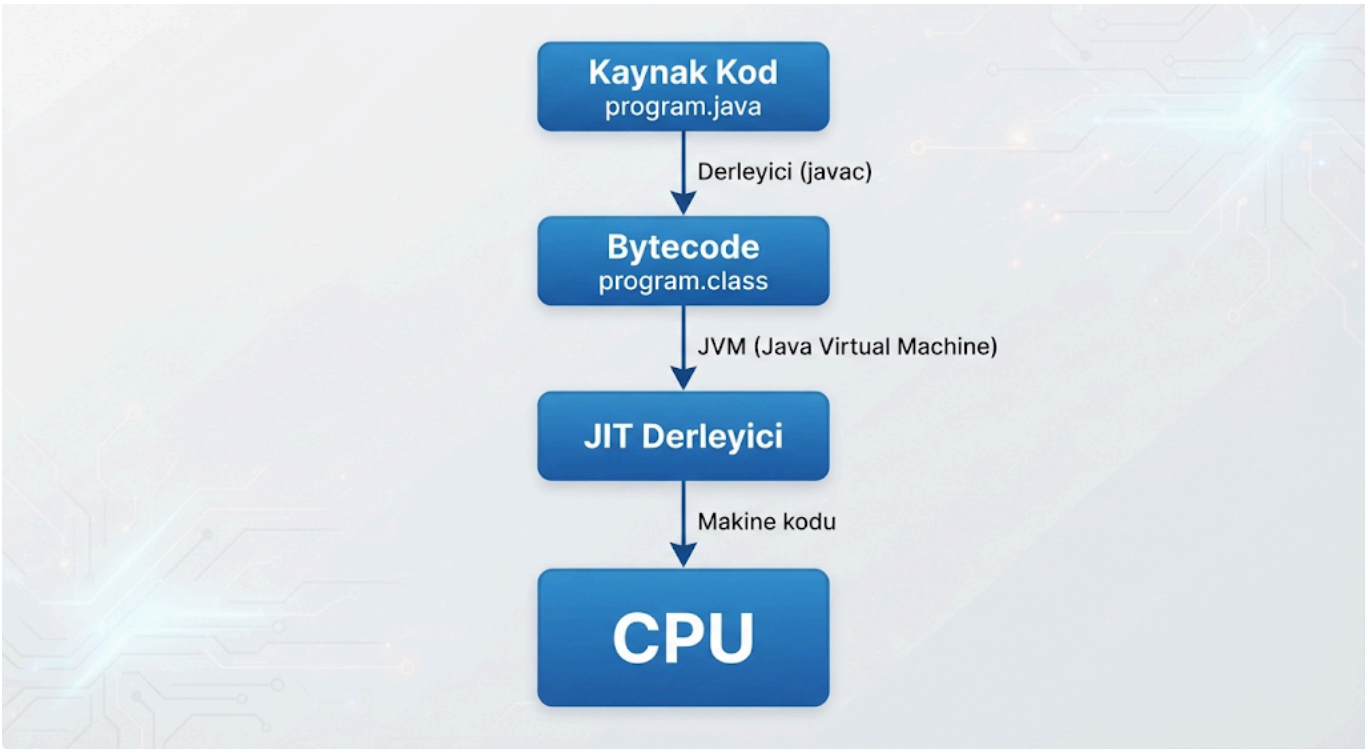
7.3 Derleyici vs Yorumlayıcı Karşılaştırması



7.4 Hibrit Yaklaşım: Bytecode ve JIT

Bazı diller hem derleyici hem yorumlayıcı yaklaşımını birleştirir.

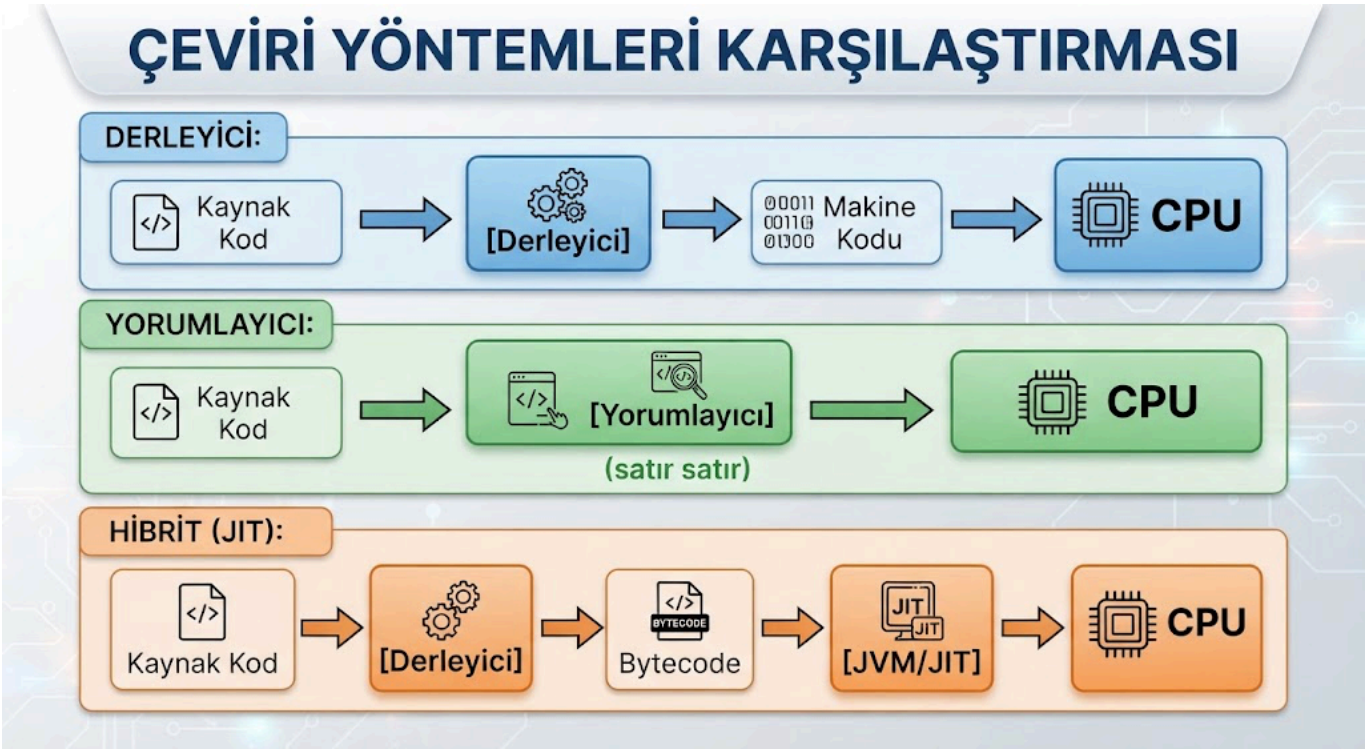
Java Örneği:



JIT (Just-In-Time) Derleyici: Bytecode'u çalışma sırasında makine koduna çevirir. Böylece hem platform bağımsızlık hem de yüksek performans sağlanır.

Örnek: Java, C#, Python (PyPy)

7.5 Çeviri Yöntemlerinin Karşılaştırmalı Şeması

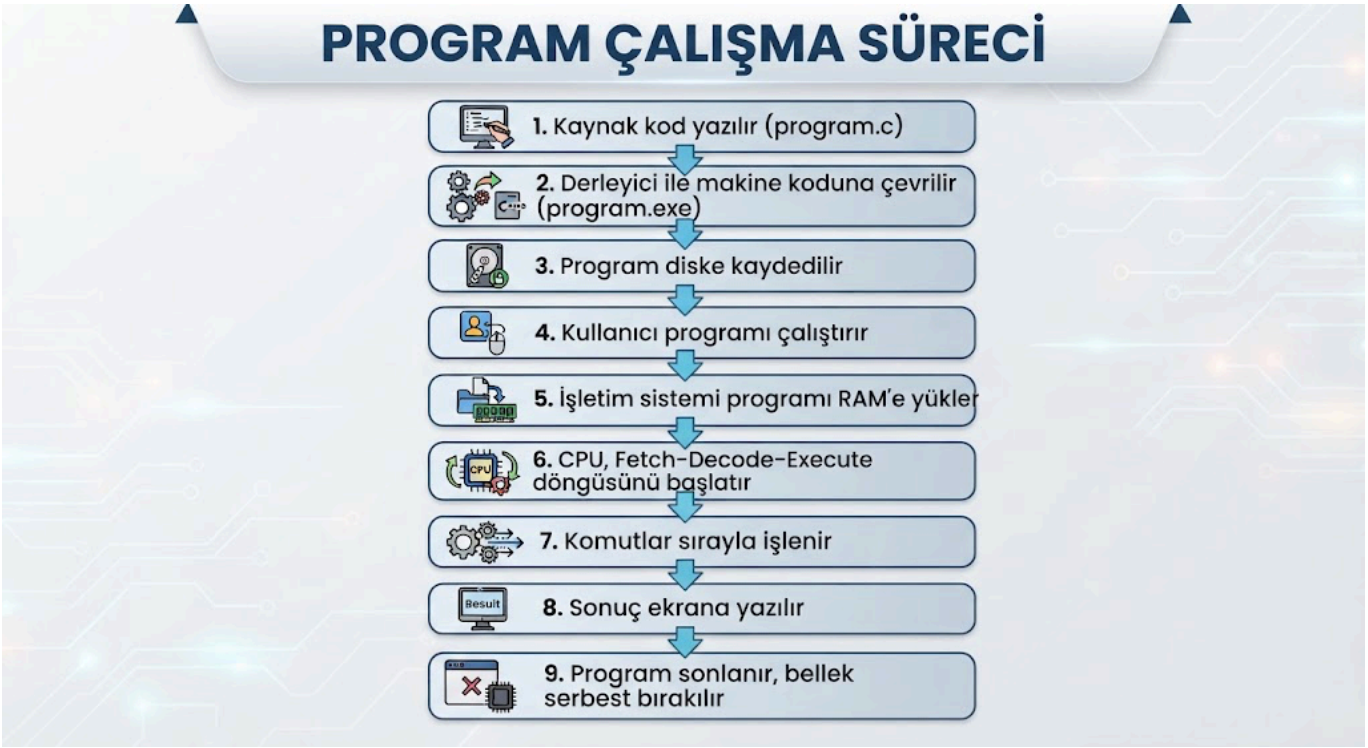


---

8. PROGRAM ÇALIŞMASI: BAŞTAN SONA

Bir programın yazılmasından çalıştırılmasına kadar geçen süreç aşağıda adım adım açıklanmıştır.

8.1 Genel Süreç



8.2 Ayrıntılı Açıklama

Adım 1: Kaynak Kod Yazımı

Programcı, bir metin editöründe kaynak kodu yazar.

```c
#include <stdio.h>

int main() {
    int a = 5;
    int b = 3;
    int toplam = a + b;
    printf("%d", toplam);
    return 0;
}
```

Adım 2: Derleme

Derleyici, kaynak kodu makine koduna çevirir.

```
gcc program.c –o program
```

Adım 3: Diske Kaydetme

Oluşturulan çalıştırılabilir dosya diske kaydedilir.

```
Disk: /home/kullanici/program
```

Adım 4: Programı Çalıştırma

Kullanıcı programı çalıştırır.

```
./program
```

Adım 5: RAM'e Yükleme

İşletim sistemi, programın makine kodunu ve gerekli verileri RAM'e yükler.

```
RAM:
+-------------------+
|  Program Kodu     |  <-- Makine komutları
+-------------------+
|  Veri Bölümü      |  <-- a=5, b=3
+-------------------+
|  Yığın (Stack)    |  <-- Fonksiyon çağrıları
+-------------------+
|  Öbek (Heap)      |  <-- Dinamik bellek
+-------------------+
```

Adım 6: Fetch–Decode–Execute Döngüsü

CPU, komutları sırayla işler.

```
1. Fetch:  PC'deki adresten komutu getir
2. Decode: Komutu çözümle
3. Execute: Komutu yürüt
4. PC'yi artır, döngüye dön
```

Adım 7: Komutların İşlenmesi

Her komut sırayla işlenir.

```
Komut 1: a = 5       -->  RAM'e 5 yaz
Komut 2: b = 3       -->  RAM'e 3 yaz
Komut 3: toplam = a+b --> ALU'da topla, sonucu yaz
Komut 4: printf      -->  Ekrana yaz
Komut 5: return 0    -->  Programı sonlandır
```

Adım 8: Çıktı

Sonuç ekrana yazılır.

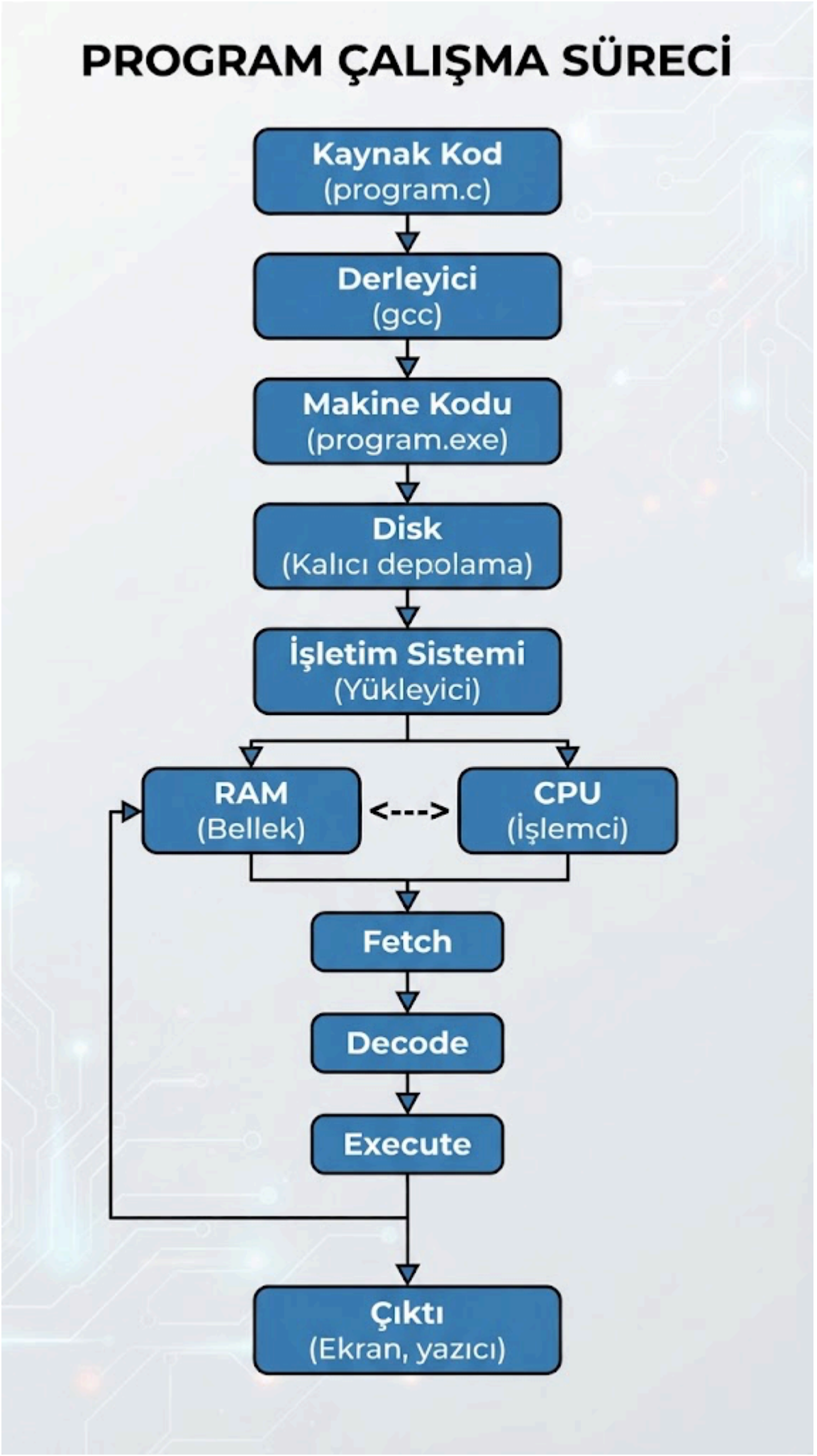
```
Ekran: 8
```

Adım 9: Sonlanma

Program sonlanır, işletim sistemi belleği serbest bırakır.

```
RAM: Programın alanı temizlenir
Disk: Program dosyası kalır
CPU: Diğer programlara geçer
```

8.3 Program Çalışmasının Bütünleşik Şeması


PROGRAM ÇALIŞMA SÜRECİ
Kaynak Kod
(program.c)
Derleyici
(gcc)
Makine Kodu
(program.exe)
Disk
(Kalıcı depolama)
İşletim Sistemi
(Yükleyici)
RAM
(Bellek)
<--->
CPU
(İşlemci)
Fetch
Decode
Execute
Çıktı
(Ekran, yazıcı)

9. ÖZET

- **Von Neumann mimarisi**, veri ve komutların aynı bellekte saklandığı modern bilgisayar mimarisidir.
- **CPU**, bilgisayarın beynidir; Kontrol Birimi, ALU, kayıtçılar ve önbellekten oluşur.
- **RAM**, çalışan programların geçici olarak saklandığı uçucu bellektir.
- **Disk**, verilerin kalıcı olarak saklandığı depolama birimidir; HDD ve SSD olmak üzere iki ana türü vardır.
- **Bellek hiyerarşisi**, hız ve kapasite arasında denge kurar: kayıtçılar > önbellek > RAM > disk.
- **Fetch–Decode–Execute döngüsü**, CPU'nun her komutu işlerken izlediği üç temel aşamadır.
- **Pipeline**, komutların paralel işlenmesini sağlayarak performansı artırır.
- **Derleyici**, kaynak kodun tamamını önceden makine koduna çevirir; **yorumlayıcı** ise satır satır çalıştırır.
- **Hibrit yaklaşımlar** (Java, C#), bytecode ve JIT derleyici ile her iki yöntemin avantajlarını birleştirir.
- Bir programın çalışması: **yazma → derleme → diske kaydetme → RAM'e yükleme → Fetch-Decode-Execute → çıktı → sonlanma** adımlarından oluşur.

10. DEĞERLENDİRME SORULARI

1. Von Neumann mimarisinin temel özelliği nedir? Açıklayınız.
2. CPU'nun temel bileşenlerini ve görevlerini yazınız.
3. RAM ile disk arasındaki farkları maddeler hâlinde belirtiniz.
4. Bellek hiyerarşisini hız ve kapasite açısından açıklayınız.
5. Fetch–Decode–Execute döngüsünün aşamalarını sırasıyla açıklayınız.
6. Pipeline kavramını örnekle açıklayınız.
7. Derleyici ile yorumlayıcı arasındaki farkları karşılaştırma tablosu hâlinde gösteriniz.
8. JIT derleyici nedir? Hangi durumda kullanılır?
9. Bir programın yazılmasından çalıştırılmasına kadar geçen süreci adım adım açıklayınız.
10. HDD ile SSD arasındaki farkları maddeler hâlinde yazınız.
11. Bir komutun CPU'da işlenme sürecini örnek bir komut üzerinden açıklayınız.
12. Önbellek (cache) nedir? Bellek hiyerarşisindeki önemi nedir?